

C. Software Comprehension Test

1. Overview

Honestly, the intent is clear and the overall structure isn't terrible for a first attempt — there's a **FrameGrabber** that listens to a native callback, a queue to buffer incoming frames, and a timer that drains the queue and reports averages. The problem is in the details: there are several correctness bugs and one fairly serious design flaw that would cause real trouble at runtime.

2. Problems in the Code

a) Shared buffer aliasing in **FrameGrabber**

This is the most critical bug

```
// FrameGrabber.FrameReceived
Marshal.Copy( frame, _buffer, 0, width * height );
Frame bufferedFrame = new Frame( _buffer );    // <-- passes the SAME array
reference
OnFrameUpdated( bufferedFrame );
```

Frame just stores the reference to **_buffer** — it doesn't copy it. So every time a new frame arrives, **Marshal.Copy** overwrites the same **_buffer** that every previously enqueued **Frame** is still pointing at. All frames in the queue end up holding a reference to the same, constantly-mutating array. By the time the timer processes them, the data inside that frame has likely already been overwritten several times by newer frames arriving from the camera native thread.

The fix if still want to use this approach : copy the buffer when constructing **Frame**:

```
byte[] copy = new byte[width * height];
Marshal.Copy( frame, copy, 0, width * height );
Frame bufferedFrame = new Frame( copy );
```

b) **Frame.Dispose()** is called immediately after enqueue

```
OnFrameUpdated( bufferedFrame );
bufferedFrame.Dispose();    // marks _disposed = true right away
```

The frame gets enqueued by **HandleFrameUpdated**, and then **Dispose()** is called on it before the timer ever has a chance to process it. So when **OnTimerElapsed** calls **frame.GetRawData()**, it will always throw **ObjectDisposedException**. The **_disposed** guard is well-intentioned but the disposal lifecycle is simply wrong.

c) Thread-safety: **Queue** is not thread-safe

`HandleFrameUpdated` is called from the native camera callback thread (or whatever thread fires `OnFrameUpdated`), while `OnTimerElapsed` runs on a `System.Timers.Timer` thread pool thread. Both access `_receivedFrames` concurrently without any synchronization. This is a classic race condition — expect occasional crashes or data corruption under load. `ConcurrentQueue<T>` would fix this with zero extra locking.

d) Integer overflow in average calculation

```
int sum = 0;
for( int i = 0; i < raw.Length; i++ )
    sum += raw[i];
int result = sum / raw.Length;
```

At 30 FPS a typical frame could be $1920 \times 1080 = \sim 2$ million pixels. Each pixel byte can be up to 255, so the sum can reach ~ 520 million — which overflows a signed 32-bit `int` (max ~ 2.1 billion). Use `long` for the accumulator. On top of that, the result is truncated to `int` before being passed to `_reporter.Report(double)`, losing all decimal precision. Should stay as `double` throughout.

e) Unnecessary complexity: the queue + timer approach

The native callback is *already* called at exactly 30 FPS by the camera driver — that's the contract. Adding a separate 30 FPS timer to drain a queue introduces drift between the two clocks and just buffers frames needlessly. If the calculation can't keep up, the queue grows without bound (no max size). The whole producer-consumer dance is unnecessary here.

f) The **IFrameCallback** interface isn't implemented properly

`FrameGrabber` receives frames via an `OnFrameUpdated` event, but `IFrameCallback.FrameReceived` takes an `IntPtr` pointing to raw memory. The intern's code never wires `FrameReceived` up to the native library — the callback is there in the interface but how the native library is told about it (e.g. via P/Invoke registration) is completely absent.

3. How I Would Reimplement It

The simplest correct approach: implement `IFrameCallback` directly, do the calculation inside `FrameReceived`, and call the reporter immediately. No queue, no timer, no aliasing.

The native library guarantees it won't reuse `pFrame` until the callback returns, so we have safe read access to the raw memory for the duration of the call.

```
using System;
using System.Runtime.InteropServices;

namespace Formulatrix.Intern.GrabTheFrame;

public interface IFrameCallback
```

```
{
    void FrameReceived( IntPtr pFrame, int width, int height );
}

public interface IValueReporter
{
    void Report( double value );
}

public class FrameCalculateAndStream : IFrameCallback
{
    private readonly IValueReporter _reporter;

    public FrameCalculateAndStream( IValueReporter reporter )
    {
        _reporter = reporter ?? throw new ArgumentNullException( nameof( reporter
    ) );
    }

    public void FrameReceived( IntPtr pFrame, int width, int height )
    {
        int totalPixels = width * height;
        if( totalPixels <= 0 )
        {
            return;
        }

        long sum = 0;

        // Read directly from unmanaged memory – no allocation needed.
        // pFrame is guaranteed valid for the duration of this callback.
        unsafe
        {
            byte* ptr = (byte*)pFrame.ToPointer();
            for( int i = 0; i < totalPixels; i++ )
            {
                sum += ptr[i];
            }
        }

        double average = (double)sum / totalPixels;
        _reporter.Report( average );
    }
}
```